

Comparative Study of Text Summarization Techniques

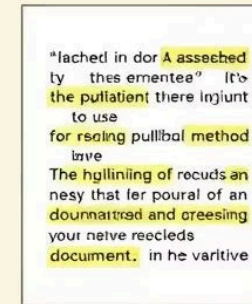
This presentation explores and compares two fundamentally different approaches to automatic text summarization: extractive and abstractive techniques. Extractive summarization works like highlighting key sentences from the original text, while abstractive summarization generates new sentences that capture core meaning in a condensed form.

We'll examine the strengths, weaknesses, and trade-offs between these methods in terms of summary quality, relevance, fluency, and computational cost through implementation, evaluation, and a live demonstration.

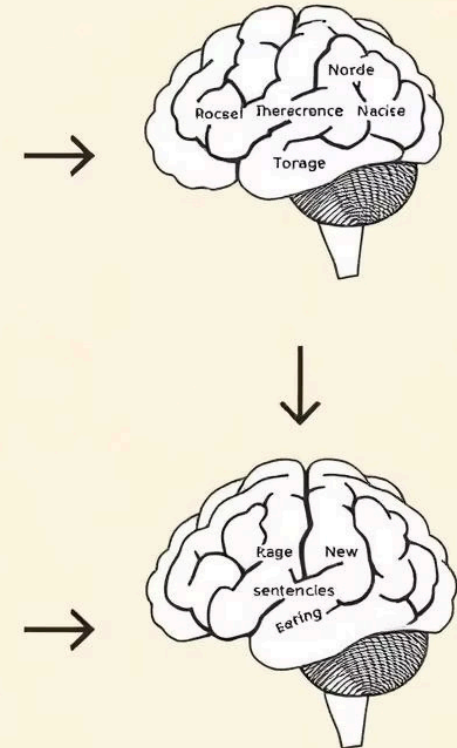
b by bhuvan

Text Summarization

Extractive method



Abstractive method



Project Overview



Project Goal

Explore, implement, and critically compare extractive and abstractive approaches to automatic text summarization



Demonstration

Showcase implementation through a live Streamlit application that allows direct comparison of different summarization techniques

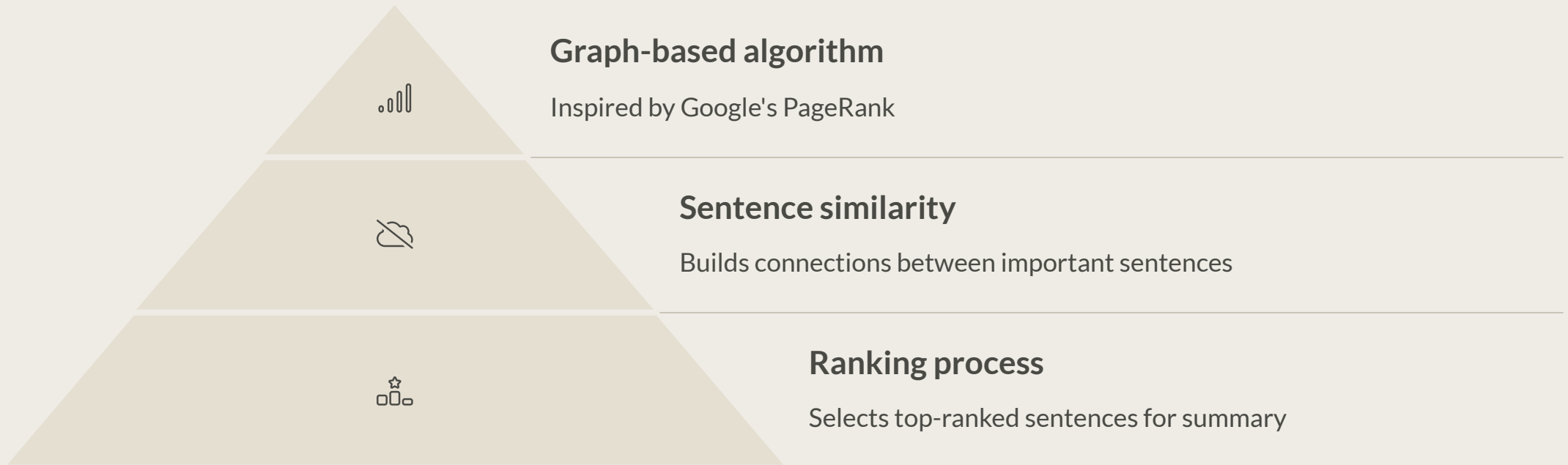


Comparison Focus

Evaluate summary quality, relevance, fluency, and computational cost between different methods



Extractive Summarization: TextRank



TextRank breaks text into sentences, calculates similarity between pairs, and builds a graph where sentences are nodes and edge weights represent similarity scores. The PageRank algorithm runs on this graph, with higher-ranked sentences selected for the summary in their original order.

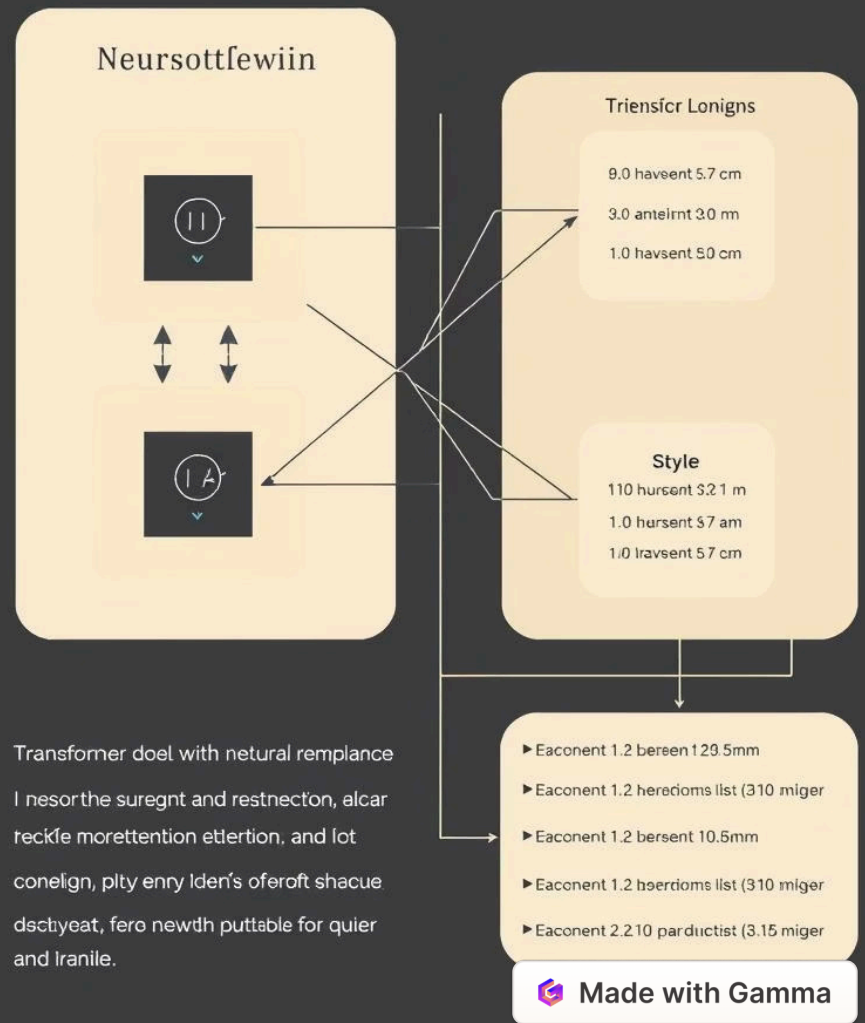
Pros: Conceptually simple, computationally fast, requires no training data, and produces factually accurate summaries. Cons: Summaries may lack coherence and can't paraphrase or synthesize information.

Abstractive Summarization: T5 & BART

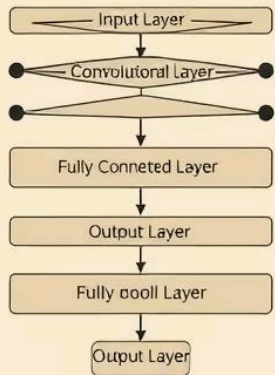


- Transformer Architecture**
Based on powerful sequence-to-sequence models with self-attention mechanisms
- Pre-training**
Models trained on massive text datasets to understand language structure
- Fine-tuning**
Pre-trained models further trained on CNN/Daily Mail dataset for summarization
- Generation**
Models produce fluent summaries with paraphrased content

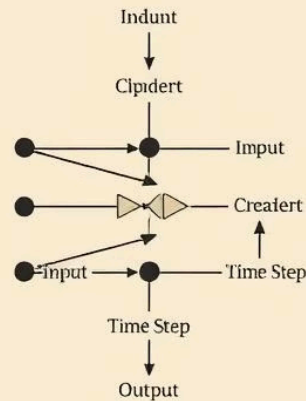
Neural Network Architecture



Acccultural sierte



Recurrurent State



Models Implemented

T5 (Text-To-Text Transfer Transformer)

Experimented with t5-small, t5-base, and t5-large to evaluate impact of model size. Fine-tuned on CNN/DM dataset using Hugging Face Trainer library with GPU/TPU resources on Kaggle.

BART (Bidirectional Auto-Regressive Transformer)

Used facebook/bart-large-cnn, a large BART model already expertly fine-tuned on CNN/Daily Mail dataset. Implemented via Hugging Face pipeline as a benchmark representing near state-of-the-art performance.

TextRank

Implemented using the sumy library in Python, which provides convenient access to this graph-based extractive algorithm. Handles sentence tokenization and ranking process efficiently.

Evaluation Strategy

Quantitative Metrics

- ROUGE Scores (ROUGE-1, ROUGE-2, ROUGE-L)
- Generation Time (computational cost)

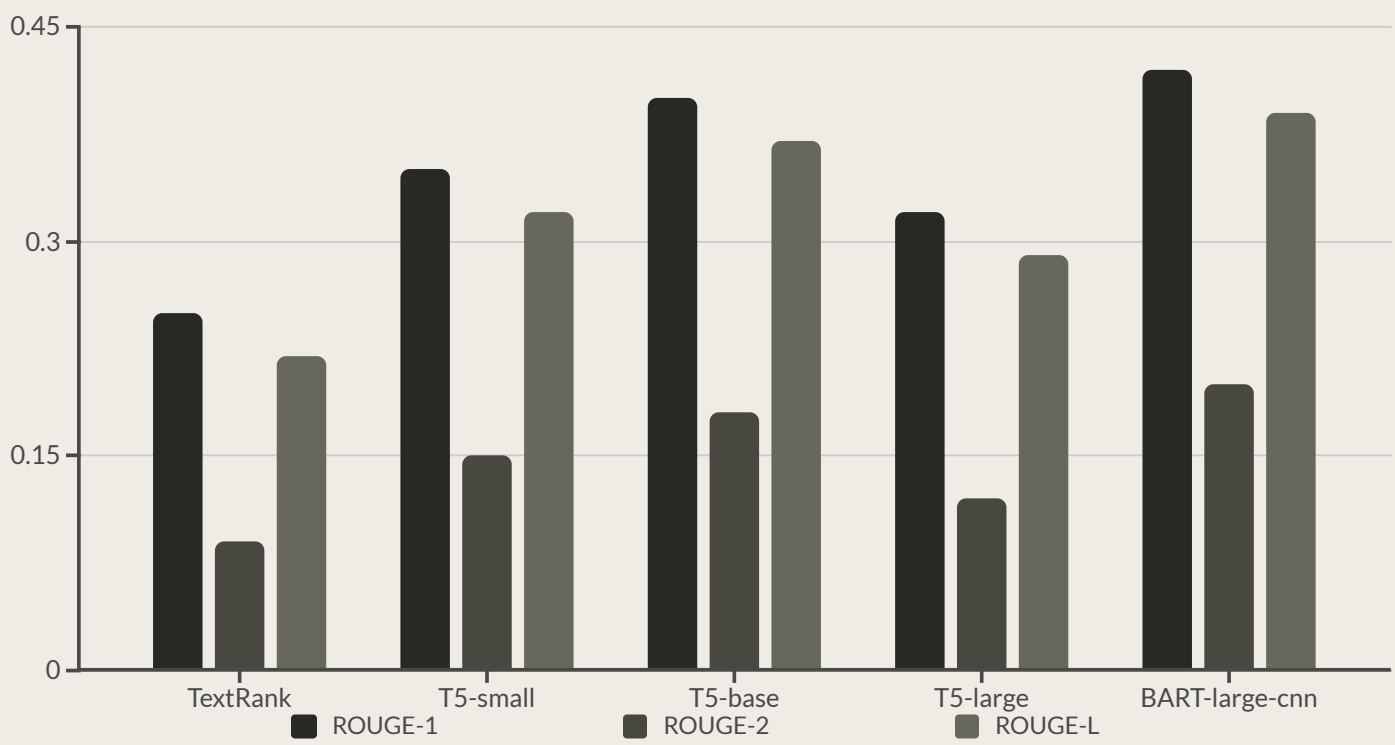
Used evaluate library wrapping rouge-score to measure overlap between generated summaries and human-written reference summaries. Higher scores indicate greater overlap.

Qualitative Assessment

- Relevance: Capture of main points
- Fluency & Coherence: Readability
- Factuality: Information accuracy
- Conciseness: Appropriate brevity

Manually reviewed summaries from each model for approximately 20 articles, comparing against original articles and reference summaries.

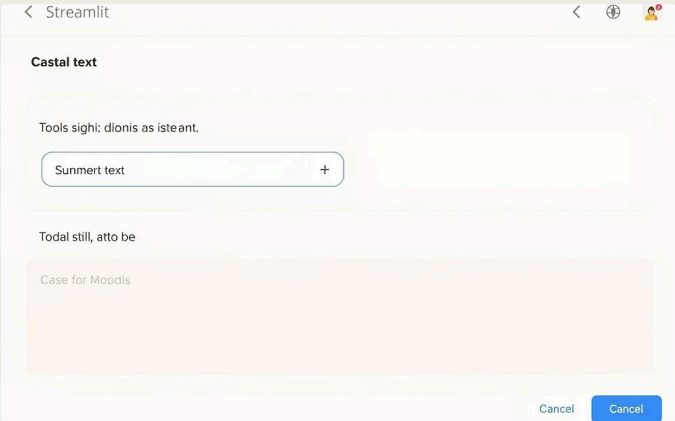
Results & Comparative Analysis



Abstractive models significantly outperformed TextRank on all ROUGE metrics. BART-large-cnn and well-trained T5-base achieved highest scores. T5-large trained on limited data (1000 samples) performed worse than T5-base despite being larger, highlighting data limitations.

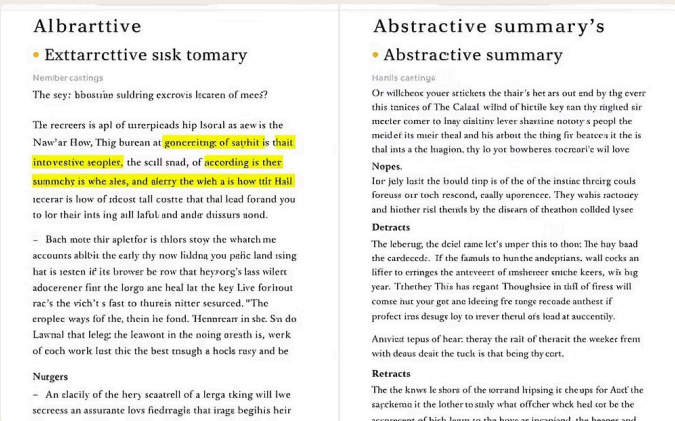
Speed comparison showed TextRank was orders of magnitude faster (~0.03s/article) than abstractive models. T5-small was ~30x slower, T5-base ~60x slower, and T5-large ~80x slower, demonstrating a stark speed-quality trade-off.

Streamlit Demo Application



User Interface

The application features an input area for text, model selection options, and parameter controls. Users can paste articles or use pre-loaded examples to generate summaries with different models.



Comparison View

Generated summaries appear in the output area with generation time displayed. This allows direct comparison between different summarization approaches and their associated trade-offs.



Live Demonstration

The demo showcases the differences in speed, fluency, and content between TextRank and transformer-based models, providing a practical illustration of the theoretical concepts.

Conclusions & Future Work



Key Findings

Well-trained abstractive models provide higher quality summaries but at higher computational cost



Critical Insights

Sufficient training data is essential for abstractive model performance



Future Directions

Fine-tune larger models, explore advanced extractive techniques, evaluate with additional metrics

This project successfully compared extractive and abstractive summarization approaches. While TextRank offers speed and simplicity, well-trained abstractive models like T5-base and BART-large-cnn provide significantly higher quality summaries in terms of fluency, coherence, and relevance, albeit at higher computational cost.

Future work could include fine-tuning larger models with full datasets, exploring advanced extractive techniques using sentence embeddings, and evaluating with metrics that assess factuality or semantic similarity more directly.